第四周课后作业标准答案

沈启立，冯赵然

第一题：主定理与矩阵乘法复杂度

大模型背景：Transformer 中的矩阵乘法

大语言模型的注意力机制（Attention）核心操作为矩阵乘法：$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$。其中 $Q,K,V \in \mathbb{R}^{n\times d}$。分析矩阵乘法算法的时间复杂度对大模型推理效率至关重要。

(a) 用主定理求解下列递推式的时间复杂度（写出所用的主定理情形）：

i. $T(n) = 8T(n/2) + O(n^2)$ （朴素矩阵乘法的分治形式）

ii. $T(n) = 7T(n/2) + O(n^2)$ （Strassen 算法）

iii. $T(n) = 4T(n/2) + O(n^2)$

iv. $T(n) = 2T(n/2) + O(n\log n)$ （归并排序变体）

(b) 在 Transformer 的自注意力中，设序列长度为 n，模型维度为 d。

i. 计算 $QK^\top$ 的朴素算法（矩阵乘法）时间复杂度是多少？

ii. 若 $d = O(n)$，朴素算法 vs Strassen 算法，哪个渐近更快？给出理由。

iii. 实践中 Strassen 算法较少用于深度学习，请给出至少一个工程原因。

标准答案：

1. 主定理求解递推式

设 $T(n) = aT(n/b) + f(n)$，$c^* = \log_b a$。

(a) $T(n) = 8T(n/2) + O(n^2)$

这里 $a = 8, b = 2 \Rightarrow c^* = \log_2 8 = 3$。

$f(n) = O(n^2) = O(n^{3-1})$，属于 **Case 1**（$f(n)$ 增长慢）。

$$T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$$

(b) $T(n) = 7T(n/2) + O(n^2)$

$a = 7, b = 2 \Rightarrow c^* = \log_2 7 \approx 2.807$

$f(n) = O(n^2) = O(n^{c^*-\epsilon})$（取 $\epsilon \approx 0.807$），属于 **Case 1**。

$$T(n) = \Theta(n^{\log_2 7})$$

(c) $T(n) = 4T(n/2) + O(n^2)$

$a = 4, b = 2 \Rightarrow c^* = \log_2 4 = 2$

$f(n) = \Theta(n^2) = \Theta(n^{c^*})$，属于 **Case 2**（$k = 0$）。

$$T(n) = \Theta(n^2 \log n)$$

(d) $T(n) = 2T(n/2) + O(n \log n)$

$a = 2, b = 2 \Rightarrow c^* = \log_2 2 = 1$

$f(n) = \Theta(n \log n) = \Theta(n^{c^*} \log^1 n)$，属于 **Case 2**（$k = 1$）。

$$T(n) = \Theta(n \log^2 n)$$

2. Attention 中矩阵乘法复杂度分析

(a) 计算 $QK^\top$

$Q \in \mathbb{R}^{n\times d}$，$K \in \mathbb{R}^{n\times d}$，因此 $K^\top \in \mathbb{R}^{d\times n}$。

矩阵乘法：

$$(n \times d) \cdot (d \times n) \Rightarrow O(n^2 d)$$

(b) 若 $d = O(n)$

朴素算法：

$$O(n^2 d) = O(n^3)$$

Strassen：

$$O(n^{\log_2 7}) \approx O(n^{2.807})$$

因此 **Strassen 渐近更快**，因为

$$2.807 < 3$$

(c) 工程中很少使用 Strassen 的原因

在理论上，Strassen 算法将矩阵乘法复杂度从 $O(n^3)$ 降低到 $O(n^{\log_2 7}) \approx O(n^{2.807})$，但在深度学习实践中（如 Transformer 中的大规模 GEMM），几乎总是采用朴素矩阵乘法（配合高度优化的 BLAS/cuBLAS）。一个关键原因可以从 **cache-oblivious** 的角度理解。

1. Strassen 算法简介（含实现方式）

将矩阵按块划分：

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

定义 7 个乘法（递归调用）：

$$\begin{aligned}
M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\
M_2 &= (A_{21} + A_{22})B_{11} \\
M_3 &= A_{11}(B_{12} - B_{22}) \\
M_4 &= A_{22}(B_{21} - B_{11}) \\
M_5 &= (A_{11} + A_{12})B_{22} \\
M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\
M_7 &= (A_{12} - A_{22})(B_{21} + B_{22})
\end{aligned}$$

组合结果：

$$\begin{aligned}
C_{11} &= M_1 + M_4 - M_5 + M_7 \\
C_{12} &= M_3 + M_5 \\
C_{21} &= M_2 + M_4 \\
C_{22} &= M_1 - M_2 + M_3 + M_6
\end{aligned}$$

实现要点：

- 递归地将矩阵划分为 $n/2 \times n/2$ 子块
- 每一层需要进行 $O(n^2)$ 次矩阵加减（产生临时矩阵）
- 通常设置一个阈值 n_0，当规模 $\leq n_0$ 时切换到朴素乘法

2. Cache-Oblivious 视角下的朴素矩阵乘法

经典三重循环矩阵乘法 $C = A \cdot B$ 可以将 A, B, C 分块为四个子矩阵，递归的重排为分治形式（cache-oblivious GEMM）：

$$C_{ij} += A_{ik} \cdot B_{kj}, \quad i, j, k \in \{1, 2\}$$

当子块大小不超过缓存容量时切换到朴素乘法。这对应 8 次子问题（但每个是连续内存块）：

$$T(n) = 8T(n/2) + O(1)$$

3. Strassen 算法的缓存行为问题

Strassen 算法虽为递归结构，但其访存模式破坏了缓存无关（cache-oblivious）算法的关键性质：

(a) **大量临时矩阵（非原位计算）**：每层需构造 $A_{11} + A_{22}$、$B_{12} - B_{22}$ 等组合，引入 $O(n^2)$ 额外内存，且写入新位置，破坏连续复用原数据的局部性。

(b) **数据复用被削弱**：朴素分治中 A_{ik}, B_{kj} 被重复使用，具有高局部性；Strassen 中同一数据块参与多个加减组合，访问顺序不规则，缓存行复用率低。

(c) **递归子问题非"干净子块"**：缓存无关 GEMM 的子问题为子矩阵乘法；Strassen 的子问题为线性组合的乘积，依赖多个原始块，增大工作集并引发更多缓存缺失。

(d) **访存复杂度更差**：在有限缓存 M 的现实硬件中，内存带宽成为瓶颈，Strassen 反而更慢。

4. 深度学习中的具体影响

以 Transformer 的矩阵乘法 $QK^\top$ 与 AV 为例：

- 数据规模大、结构规则，适合 SIMD/GPU 的高吞吐计算；
- Strassen 破坏 GEMM 的规则访存模式，难以适配 cuBLAS 等优化库，增加内存流量（加减操作引入额外读写），降低 GPU 张量核心利用率。

Strassen 虽减少乘法次数，但破坏缓存局部性，增加内存访问成本。朴素分治 GEMM 是缓存无关最优的；Strassen 引入大量临时数据与不规则访存。在内存受限的现代硬件上，内存成本远高于算术成本。因此深度学习中优化访存（缓存/带宽）比减少乘法次数更为关键。

(a) 用主定理求解递推式 **i.** $T(n) = 8T(n/2) + O(n^2)$

参数识别：$a = 8,\ b = 2,\ f(n) = O(n^2)$。

临界指数：$c^* = \log_2 8 = 3$。

比较：$f(n) = O(n^2) = O(n^{3-1})$，即 $f(n) = O(n^{c^*-\epsilon})$，$\epsilon = 1$。

$\Rightarrow$ **Case 1**（递归调用主导）。

$$\boxed{T(n) = \Theta(n^3)}$$

ii. $T(n) = 7T(n/2) + O(n^2)$

参数识别：$a = 7,\ b = 2,\ f(n) = O(n^2)$。
临界指数：$c^* = \log_2 7 \approx 2.807$。
比较：$f(n) = O(n^2) = O(n^{2.807-0.807})$。
$\Rightarrow$ **Case 1**。

$$\boxed{T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})}$$

iii. $T(n) = 4T(n/2) + O(n^2)$
参数识别：$a = 4,\ b = 2,\ f(n) = O(n^2)$。
临界指数：$c^* = \log_2 4 = 2$。
比较：$f(n) = \Theta(n^2) = \Theta(n^{c^*})$。
$\Rightarrow$ **Case 2**，$k = 0$。

$$\boxed{T(n) = \Theta(n^2 \log n)}$$

iv. $T(n) = 2T(n/2) + O(n \log n)$
参数识别：$a = 2,\ b = 2,\ f(n) = O(n \log n)$。
临界指数：$c^* = \log_2 2 = 1$。
比较：$f(n) = \Theta(n^1 \cdot \log^1 n) = \Theta(n^{c^*} \log^k n)$，$k = 1$。
$\Rightarrow$ **Case 2**，$k = 1$。

$$\boxed{T(n) = \Theta(n \log^2 n)}$$

(b) Transformer 自注意力中的矩阵乘法 **i.** $Q \in \mathbb{R}^{n\times d}, K \in \mathbb{R}^{n\times d}$, 计算 $QK^\top \in \mathbb{R}^{n\times n}$。

结果矩阵有 n^2 个元素，每个元素是两个 d 维向量的内积，需要 $O(d)$ 次标量乘法和加法。

$$\boxed{\text{朴素矩阵乘法时间复杂度为}O(n^2 d)}$$

ii. 若 $d = O(n)$，则 $Q, K \in \mathbb{R}^{n\times O(n)}$，即两个 $n \times n$ 量级的矩阵相乘。

- 朴素算法：$O(n^2 d) = O(n^3)$
- Strassen 算法：$O(n^{\log_2 7}) \approx O(n^{2.807})$

Strassen 渐近更快，因为 $n^{2.807} = o(n^3)$。
iii. Strassen 较少用于深度学习的工程原因：

- **数值稳定性差**：Strassen 涉及大量加减运算，浮点误差累积显著，而深度学习中广泛使用低精度（FP16/BF16），误差问题更为突出。

- **GPU 硬件优化**：现代 GPU 的 Tensor Core 专门针对朴素矩阵乘法（GEMM）进行硬件加速，朴素算法的实际吞吐量极高，Strassen 的理论优势被硬件优化抵消。
- **常数因子与额外内存开销**：Strassen 需要更多中间矩阵的分配和加减操作，常数因子大，仅在矩阵规模极大时才有实际优势。

第二题：动态规划——最长公共子序列（LCS）与代码相似度检测

大模型背景：LLM 生成代码的抄袭/泄露检测

大语言模型生成的代码可能与训练数据中的代码高度相似，引发版权和隐私风险。**最长公共子序列（LCS）**是衡量两段序列结构相似度的经典方法：将代码 token 化后，计算生成序列与训练样本的 LCS 长度，可作为”记忆泄露”（memorization）程度的量化指标。

(a) 设 $dp[i][j]$ 为序列 $X[1..i]$ 与 $Y[1..j]$ 的最长公共子序列长度。写出：

i. 状态转移方程（分 $X[i] = Y[j]$ 和 $X[i] \neq Y[j]$ 两种情况讨论）

ii. 边界条件 $dp[i][0]$ 和 $dp[0][j]$

iii. 时间复杂度与空间复杂度

(b) 设某 LLM 生成的 token 序列为 $X = [\texttt{def}, \texttt{sort}, \texttt{list}, \texttt{return}, \texttt{sorted}, \texttt{list}]$（长度 6），训练集中的一段代码 token 序列为 $Y = [\texttt{def}, \texttt{merge}, \texttt{sort}, \texttt{return}, \texttt{list}]$（长度 5）。

手动填写完整的 DP 表格（7×6，含边界行列），给出 LCS 长度。

$dp[i][j]$	ε	def	merge	sort	return	list
ε	0	0	0	0	0	0
def	0					
sort	0					
list	0					
return	0					
sorted	0					
list	0					

回溯找出一个具体的 LCS，并计算相似度指标 $\text{sim} = \dfrac{2\,|\text{LCS}|}{|X| + |Y|}$。

(c) 用**剪切-粘贴论证法**证明 LCS 的最优子结构：

核心要求：待证命题

设 $Z = z_1 z_2 \cdots z_k$ 是 $X[1..m]$ 与 $Y[1..n]$ 的一个 LCS。若 $x_m = y_n$，则 $z_k = x_m = y_n$，且 $Z[1..k-1]$ 是 $X[1..m-1]$ 与 $Y[1..n-1]$ 的 LCS。

提示：先证 $z_k = x_m$（反证：若不是，可以将 x_m 附加到 Z 末尾得到更长公共子序列）；再证 $Z[1..k-1]$ 是最优的（若存在更长的公共子序列 W，可以粘贴 x_m 得到矛盾）。

标准答案：

(a) LCS 的状态转移方程 **i. 状态转移方程**：

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{若} X[i] = Y[j] \\ \max\big(dp[i-1][j],\ dp[i][j-1]\big) & \text{若} X[i] \neq Y[j] \end{cases}$$

ii. 边界条件：

$$dp[i][0] = 0, \quad \forall\, 0 \le i \le m; \qquad dp[0][j] = 0, \quad \forall\, 0 \le j \le n$$

空序列与任何序列的 LCS 长度为 0。

iii. 复杂度：

- 时间复杂度：$O(mn)$（需填写 $m \times n$ 的表格，每格 $O(1)$）
- 空间复杂度：$O(mn)$（存储整张 DP 表；可优化为 $O(\min(m,n))$ 只保留两行）

(b) 手动填写 DP 表格 $X = [\texttt{def}, \texttt{sort}, \texttt{list}, \texttt{return}, \texttt{sorted}, \texttt{list}], Y = [\texttt{def}, \texttt{merge}, \texttt{sort}, \texttt{return}, \texttt{l}$

逐行填写：

$dp[i][j]$	ε	def	merge	sort	return	list
ε	0	0	0	0	0	0
def	0	1	1	1	1	1
sort	0	1	1	2	2	2
list	0	1	1	2	2	3
return	0	1	1	2	3	3
sorted	0	1	1	2	3	3
list	0	1	1	2	3	4

LCS 长度 $= dp[6][5] = \boxed{4}$。

回溯找 LCS：从 $dp[6][5] = 4$ 出发，沿箭头回溯：

- $dp[6][5] = 4$：$X[6] = \texttt{list} = Y[5]$，取 `list`，转到 $dp[5][4]$
- $dp[5][4] = 3$：$X[5] = \texttt{sorted} \neq Y[4] = \texttt{return}$，$dp[4][4] = 3 > dp[5][3] = 2$，转到 $dp[4][4]$
- $dp[4][4] = 3$：$X[4] = \texttt{return} = Y[4]$，取 `return`，转到 $dp[3][3]$
- $dp[3][3] = 2$：$X[3] = \texttt{list} \neq Y[3] = \texttt{sort}$，$dp[2][3] = 2 > dp[3][2] = 1$，转到 $dp[2][3]$
- $dp[2][3] = 2$：$X[2] = \texttt{sort} = Y[3]$，取 `sort`，转到 $dp[1][2]$
- $dp[1][2] = 1$：$X[1] = \texttt{def} \neq Y[2] = \texttt{merge}$，$dp[1][1] = 1 > dp[0][2] = 0$，转到 $dp[1][1]$
- $dp[1][1] = 1$：$X[1] = \texttt{def} = Y[1]$，取 `def`，转到 $dp[0][0] = 0$，结束

一个具体的 LCS：[`def`, `sort`, `return`, `list`]，长度为 4。

相似度指标：

$$\text{sim} = \frac{2\,|\text{LCS}|}{|X| + |Y|} = \frac{2 \times 4}{6 + 5} = \boxed{\frac{8}{11} \approx 0.727}$$

(c) 剪切-粘贴论证法证明 LCS 最优子结构

证明. 设 $Z = z_1 z_2 \cdots z_k$ 是 $X[1..m]$ 与 $Y[1..n]$ 的一个 LCS，且 $x_m = y_n$。

第一步：证明 $z_k = x_m = y_n$。

反证法：假设 $z_k \neq x_m$。由于 Z 是 X 的子序列且 $z_k \neq x_m = X[m]$，Z 中所有元素的匹配位置都在 $X[1..m{-}1]$ 中。同理，由于 $z_k \neq y_n$，Z 的匹配位置也都在 $Y[1..n{-}1]$ 中。

因此，可以将 x_m（$= y_n$）附加到 Z 的末尾，得到 $Z' = z_1 z_2 \cdots z_k\, x_m$。

- Z' 是 $X[1..m]$ 的子序列：$z_1 \cdots z_k$ 匹配 $X[1..m{-}1]$，x_m 匹配 $X[m]$。
- Z' 是 $Y[1..n]$ 的子序列：$z_1 \cdots z_k$ 匹配 $Y[1..n{-}1]$，y_n 匹配 $Y[n]$。

Z' 是长度为 $k+1$ 的公共子序列，与 Z 是 LCS（长度 k）矛盾。故 $z_k = x_m = y_n$。

第二步：证明 $Z[1..k{-}1]$ **是** $X[1..m{-}1]$ **与** $Y[1..n{-}1]$ **的 LCS**。

由第一步，$z_k = x_m = y_n$，因此 $Z[1..k{-}1] = z_1 \cdots z_{k-1}$ 是 $X[1..m{-}1]$ 与 $Y[1..n{-}1]$ 的公共子序列。

反证法：假设 $Z[1..k{-}1]$ 不是 $X[1..m{-}1]$ 与 $Y[1..n{-}1]$ 的 LCS。则存在更长的公共子序列 W，满足 $|W| \geq k$。

剪切：从 Z 中移除 $Z[1..k{-}1]$。**粘贴**：用 W 替换，得到 $W' = W \cdot x_m$。

W' 是 $X[1..m]$ 与 $Y[1..n]$ 的公共子序列（W 匹配 $X[1..m{-}1]$ 和 $Y[1..n{-}1]$，$x_m = y_n$ 匹配末尾），且 $|W'| = |W| + 1 \geq k + 1 > k = |Z|$。

这与 Z 是 LCS 矛盾。因此 $Z[1..k{-}1]$ 是 $X[1..m{-}1]$ 与 $Y[1..n{-}1]$ 的 LCS。 □

第三题：分治法——用 FFT 加速卷积

大模型背景：卷积神经网络（CNN）中的卷积加速

卷积神经网络的核心操作是一维或二维离散卷积。朴素卷积的时间复杂度为 $O(n^2)$，而利用分治法和快速傅里叶变换（FFT）可将其降至 $O(n \log n)$，这在处理长序列（如语音、DNA 序列）时至关重要。

(a) 两个长度为 n 的多项式（序列）的卷积朴素算法为 $O(n^2)$。Cooley-Tukey FFT 算法采用分治策略，将 DFT 拆分为奇偶两部分：

$$F[k] = F_{\text{even}}[k] + e^{-2\pi ik/n} F_{\text{odd}}[k]$$

写出该算法的递推式，并用主定理求解其时间复杂度。

(b) 对长度 $n = 8$ 的序列，画出 FFT 蝴蝶操作的递归结构示意图（树形图），标注每一层的操作数量，并说明：第 ℓ 层（$\ell = 1, 2, \ldots, \log_2 n$）共有多少个独立的蝴蝶操作？

(c) 在 GPU 上并行执行 FFT 时：

i. 每一层的蝴蝶操作可以完全并行，假设 GPU 有足够的并行核心（$\geq n/2$），分析 GPU 上 FFT 的并行时间复杂度（按层数计）。

ii. 相比 CPU 上 $O(n \log n)$ 的顺序时间，GPU 上的加速比是多少？

iii. 用 FFT 加速 CNN 卷积时，实际并行效率受哪些因素影响？请列举两点。

标准答案：

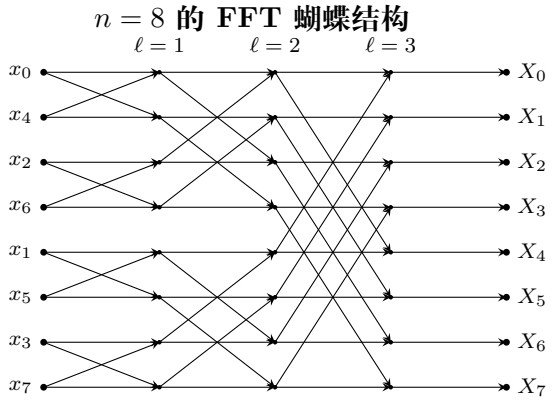
(a) FFT 的递推式与复杂度 Cooley-Tukey FFT 将长度 n 的 DFT 分解为两个长度 $n/2$ 的 DFT（偶数项和奇数项），然后用 $O(n)$ 次蝴蝶操作合并。递推式为：

$$T(n) = 2T(n/2) + O(n)$$

用主定理：$a = 2$, $b = 2$, $f(n) = O(n)$。临界指数 $c^* = \log_2 2 = 1$，$f(n) = \Theta(n^{c^*})$，属于 Case 2（$k = 0$）。

$$\boxed{T(n) = \Theta(n \log n)}$$

(b) FFT 蝴蝶操作的递归结构 对 $n = 8$ 的序列，FFT 的蝴蝶结构有 $\log_2 8 = 3$ 层（stage）。



每一层的蝴蝶操作数和特征：

层 ℓ	蝴蝶组数	每组蝴蝶数	独立蝴蝶操作总数
1	4 组（各 2 点）	1	4
2	2 组（各 4 点）	2	4
3	1 组（8 点）	4	4

一般地，第 ℓ 层共有 $\boxed{n/2}$ 个独立的蝴蝶操作（每个蝴蝶操作涉及一对元素的加减和旋转因子乘法）。

(c) GPU 上的 FFT 并行分析 **i.** 每一层有 $n/2$ 个相互独立的蝴蝶操作，可完全并行执行。假设 GPU 有 $\geq n/2$ 个并行核心，每层的执行时间为 $O(1)$。共有 $\log_2 n$ 层，因此：

$$\boxed{T_{\text{GPU}}(n) = O(\log n)}$$

ii. 加速比：

$$\text{Speedup} = \frac{T_{\text{CPU}}}{T_{\text{GPU}}} = \frac{O(n \log n)}{O(\log n)} = \boxed{O(n)}$$

iii. 实际并行效率受以下因素影响：

- **内存带宽与访存模式**：FFT 的蝴蝶操作涉及 bit-reversal 置换和不同步长的数据访问，导致非连续内存访问，GPU 的显存带宽可能成为瓶颈，cache 利用率低。
- **层间同步开销**：不同层之间存在数据依赖，必须在每层结束后进行线程同步（如 `__syncthreads()`），同步开销在层数较多时累积显著。

第四题：动态规划——矩阵链乘法与最优括号化

大模型背景：Transformer 中的多矩阵乘法顺序优化

Transformer 推理时，一个注意力头的计算为 $((QK^\top)V)W_O$，涉及多个矩阵连乘。不同的括号化方式（计算顺序）会导致截然不同的计算量。对矩阵链乘法顺序进行优化，可显著降低推理时间。

(a) 写出矩阵链乘法的 DP 状态定义和转移方程：设 $m[i,j]$ 为计算矩阵 $A_iA_{i+1}\cdots A_j$ 所需的最少标量乘法次数，写出 $m[i,j]$ 的递推式（$i<j$ 时），以及时间复杂度。

(b) 给定如下 4 个矩阵（维度模拟 Transformer 注意力计算）：

$$A_1 \in \mathbb{R}^{64\times 512},\quad A_2 \in \mathbb{R}^{512\times 64},\quad A_3 \in \mathbb{R}^{64\times 512},\quad A_4 \in \mathbb{R}^{512\times 1}$$

i. 用 DP 方法求计算 $A_1A_2A_3A_4$ 的**最优括号化方式**及最少标量乘法次数。请填写 $m[i,j]$ 的完整表格。

ii. 列举所有可能的括号化方式中，**最差**的括号化方式需要多少次标量乘法？与最优方案相差多少倍？

标准答案：

(a) 矩阵链乘法的 DP 状态与转移方程 设矩阵 A_i 的维度为 $p_{i-1}\times p_i$。定义 $m[i,j]$ 为计算 $A_iA_{i+1}\cdots A_j$ 所需的最少标量乘法次数。

$$m[i,j] = \begin{cases} 0 & \text{若} i=j \\ \min\limits_{i\le k<j}\left\{m[i,k]+m[k{+}1,j]+p_{i-1}\cdot p_k\cdot p_j\right\} & \text{若} i<j \end{cases}$$

时间复杂度：三重循环（链长、起点、分割点），为 $O(n^3)$。

(b) 手算 维度参数：$p_0=64,\ p_1=512,\ p_2=64,\ p_3=512,\ p_4=1$。

i. 填写 $m[i,j]$ 表格：

基础（链长 1）：$m[1,1]=m[2,2]=m[3,3]=m[4,4]=0$

链长 2：

$$m[1,2]=p_0p_1p_2=64\times 512\times 64=2{,}097{,}152$$
$$m[2,3]=p_1p_2p_3=512\times 64\times 512=16{,}777{,}216$$
$$m[3,4]=p_2p_3p_4=64\times 512\times 1=32{,}768$$

链长 3：

$$m[1,3] = \min \begin{cases} k{=}1: \ m[1,1] + m[2,3] + p_0p_1p_3 = 0 + 16{,}777{,}216 + 16{,}777{,}216 = 33{,}554{,}432 \\ k{=}2: \ m[1,2] + m[3,3] + p_0p_2p_3 = 2{,}097{,}152 + 0 + 2{,}097{,}152 = 4{,}194{,}304 \end{cases}$$
$$= 4{,}194{,}304 \quad (k^* = 2)$$

$$m[2,4] = \min \begin{cases} k{=}2: \ m[2,2] + m[3,4] + p_1p_2p_4 = 0 + 32{,}768 + 32{,}768 = 65{,}536 \\ k{=}3: \ m[2,3] + m[4,4] + p_1p_3p_4 = 16{,}777{,}216 + 0 + 262{,}144 = 17{,}039{,}360 \end{cases}$$
$$= 65{,}536 \quad (k^* = 2)$$

链长 4：

$$m[1,4] = \min \begin{cases} k{=}1: \ m[1,1] + m[2,4] + p_0p_1p_4 = 0 + 65{,}536 + 32{,}768 = 98{,}304 \\ k{=}2: \ m[1,2] + m[3,4] + p_0p_2p_4 = 2{,}097{,}152 + 32{,}768 + 4{,}096 = 2{,}134{,}016 \\ k{=}3: \ m[1,3] + m[4,4] + p_0p_3p_4 = 4{,}194{,}304 + 0 + 32{,}768 = 4{,}227{,}072 \end{cases}$$
$$= 98{,}304 \quad (k^* = 1)$$

$m[i,j]$	$j=1$	$j=2$	$j=3$	$j=4$
$i=1$	0	2,097,152	4,194,304	**98,304**
$i=2$		0	16,777,216	65,536
$i=3$			0	32,768
$i=4$				0

最优括号化方式：$m[1,4]$ 在 $k=1$ 处取得最小值，即 $(A_1)\big((A_2)(A_3A_4)\big)$；$m[2,4]$ 在 $k=2$ 处取得最小值，即 $(A_2)(A_3A_4)$。因此最优括号化为：

$$\boxed{A_1 \cdot \big(A_2 \cdot (A_3 \cdot A_4)\big), \quad \text{最少标量乘法次数} = 98{,}304}$$

验证：A_3A_4 $(64{\times}512{\cdot}512{\times}1 = 64{\times}1$，代价 32,768$) \to A_2(A_3A_4)$ $(512{\times}64{\cdot}64{\times}1 = 512{\times}1$，代价 32,768$) \to A_1(\cdots)$ $(64{\times}512{\cdot}512{\times}1 = 64{\times}1$，代价 32,768$)$。总计 $3{\times}32{,}768 = 98{,}304$。✓

ii. 最差括号化方式：

4 个矩阵共有 $C_3 = 5$ 种括号化方式，各方案代价如下：

编号	括号化方式	标量乘法次数
1	$\big((A_1A_2)A_3\big)A_4$	4,227,072
2	$\big(A_1(A_2A_3)\big)A_4$	33,587,200
3	$(A_1A_2)(A_3A_4)$	2,134,016
4	$A_1\big((A_2A_3)A_4\big)$	17,072,128
5	$A_1\big(A_2(A_3A_4)\big)$	98,304

最差方案为第 2 种 $\big(A_1(A_2A_3)\big)A_4$，需要 $\boxed{33{,}587{,}200}$ 次标量乘法。
与最优方案的比值：$33{,}587{,}200 \,/\, 98{,}304 \approx \boxed{342}$ 倍。
这说明括号化顺序对多矩阵连乘的计算量影响极其显著。

第五题：状态压缩 DP——Beam Search 路径规划

大模型背景：语言模型解码：Beam Search vs 贪心解码

语言模型自回归生成时，每步从词表 V 中选一个 token。**贪心解码**每步选概率最高的 token；**Beam Search** 保留 B 条候选路径，以期找到全局更优的序列。理解这两种解码策略的本质，需要动态规划的视角。

(a) 考虑一个简化问题：词表大小为 n，生成长度为 T 的序列，每步的对数概率为 $\log p(\text{token}_t|\text{context})$。

i. 将”找最大对数概率序列”形式化为 DP 问题，定义状态和转移方程。

ii. 暴力枚举所有长度为 T 的序列的时间复杂度是多少？DP 能否将其降低？为什么？（提示：注意这里的语言模型是自回归的，即 $p(x_t|x_{<t})$ 依赖全部历史。）

(b) 考虑如下反例：词表 $V = \{A, B, C\}$，生成长度 $T = 3$，各步概率如下表：

步骤	第 1 步			第 2 步（上步选 A）			第 3 步（前两步 AA）		
Token	A	B	C	A	B	C	A	B	C
概率	0.5	0.3	0.2	0.1	0.8	0.1	0.9	0.05	0.05
				第 2 步（上步选 B）			第 3 步（前两步 BA）		
Token				A	B	C	A	B	C
概率				0.6	0.3	0.1	0.1	0.8	0.1

i. 贪心解码给出的序列是什么？总概率是多少？

ii. 通过枚举，找到概率最高的序列（长度 3），证明贪心解码**不一定**给出全局最优解。

(c) 经典 **TSP 状压 DP**（旅行商问题）：n 个城市，$dp[S][v]$ 表示从城市 0 出发，已访问城市集合为 S，当前在城市 v 的最短路径长度。

i. 写出 $dp[S][v]$ 的状态转移方程。

ii. 状态总数和总时间复杂度各是多少？

iii. 若每个城市代表一个候选解码步骤，这与 Beam Search 有何结构上的相似性？简要说明（3 句话以内）。

标准答案：

(a) 将最大对数概率序列形式化为 DP 问题 i. 状态定义与转移方程：

定义状态 $dp[t][h_t]$ 为生成前 t 个 token 的最大累积对数概率，其中 $h_t = (x_1, x_2, \ldots, x_t)$ 为完整的前缀历史。

转移方程：

$$dp[t][h_t] = \max_{x_t \in V} \Big\{ dp[t-1][h_{t-1}] + \log p(x_t \mid h_{t-1}) \Big\}$$

其中 $h_t = (h_{t-1}, x_t)$。初始条件：$dp[0][\varepsilon] = 0$（空前缀的累积对数概率为 0）。

最终目标：$\max_{h_T} dp[T][h_T]$。

ii. 暴力枚举的时间复杂度：

暴力枚举所有长度为 T 的序列，共 n^T 种，时间复杂度为 $O(n^T)$。

DP **无法**将其有效降低。原因在于自回归语言模型的条件概率 $p(x_t \mid x_{<t})$ 依赖于**完整的历史**（所有前面的 token），不同的历史产生不同的条件概率分布。因此，状态空间为所有可能的前缀 h_t，共有 $\sum_{t=0}^{T} n^t = O(n^T)$ 个不同状态，DP 无法合并这些状态。这正是 Beam Search 只保留 B 条最优路径来做近似的原因。

(b) 贪心解码的反例 i. 贪心解码：

- 第 1 步：$\max\{0.5, 0.3, 0.2\}$，选 A，概率 0.5
- 第 2 步（上步选 A）：$\max\{0.1, 0.8, 0.1\}$，选 B，概率 0.8
- 第 3 步（前两步 AB）：表中未提供该前缀的条件概率

贪心在第 1 步锁定了 A（局部最优），但这导致后续路径受限。在表中给出完整数据的路径中，从 A 出发能达到的最佳完整序列为 $A \to A \to A$，总概率 $= 0.5 \times 0.1 \times 0.9 = 0.045$。

ii. 通过枚举找全局最优：

在表中给出完整数据的所有路径中：

序列	总概率 $P(x_1)P(x_2\|x_1)P(x_3\|x_1x_2)$
AAA	$0.5 \times 0.1 \times 0.9 = 0.045$
AAB	$0.5 \times 0.1 \times 0.05 = 0.0025$
AAC	$0.5 \times 0.1 \times 0.05 = 0.0025$
BAA	$0.3 \times 0.6 \times 0.1 = 0.018$
BAB	$\mathbf{0.3 \times 0.6 \times 0.8 = 0.144}$
BAC	$0.3 \times 0.6 \times 0.1 = 0.018$

概率最高的序列是 $\boxed{BAB}$，总概率 $= \boxed{0.144}$。

贪心在第 1 步选了 A（概率 $0.5 > 0.3$），从而错过了全局最优路径 $B \to A \to B$。虽然 B 的第 1 步概率较低（0.3），但后续的高条件概率（0.6 和 0.8）使得总概率远超从 A 出发的最佳路径（0.045）。这证明了**贪心解码不一定给出全局最优解**。

(c) TSP 状压 DP　i. 状态转移方程：

$dp[S][v]$ 表示从城市 0 出发，已访问的城市集合为 S（包含城市 0 和 v），当前位于城市 v 的最短路径长度。

$$dp[S][v] = \min_{\substack{u \in S,\, u \neq v \\ (u,v) \in E}} \big\{ dp[S \setminus \{v\}][u] + w(u,v) \big\}$$

边界条件：$dp[\{0\}][0] = 0$。

最终答案：$\min_{v \neq 0} \big\{ dp[\{0,1,\ldots,n-1\}][v] + w(v,0) \big\}$。

ii. 状态总数与时间复杂度：

- 状态总数：$2^n \times n$（每个子集 $S \subseteq \{0,\ldots,n-1\}$ 和每个当前城市 v）。
- 每个状态的转移需要枚举 $O(n)$ 个前驱城市。
- 总时间复杂度：$\boxed{O(2^n \cdot n^2)}$。

iii. 与 Beam Search 的结构相似性：

TSP 状压 DP 和 Beam Search 都维护" 部分路径" 并逐步扩展：每个状态记录已走过的历史（TSP 中是已访问的城市集合，Beam Search 中是已生成的 token 前缀）以及当前位置，并通过动态规划寻找全局最优路径。Beam Search 本质上是对完整 DP 搜索空间的启发式剪枝——每步只保留得分最高的 B 条路径，而非遍历所有 n^t 种可能的前缀。

第六题：KV Cache 页分配——前缀共享下的树形背包 DP

大模型背景：vLLM 的 PagedAttention 与前缀共享

大模型推理引擎（如 vLLM）使用 **PagedAttention**：将 KV Cache 分成固定大小的**物理页**（block），按需分配给各请求。当多个请求共享**相同的前缀**（如相同的 system prompt）时，共享前缀对应的 KV Cache 页只需存储**一份**，多个请求可共用——这称为 **prefix sharing**（前缀共享），是 vLLM 的关键优化之一。在物理页总量有限时，如何选择服务哪些请求、利用前缀共享节省页数，可以建模为一个**树形依赖背包问题**。

问题形式化

系统中有若干推理请求，它们的 prompt 前缀关系构成一棵**前缀树**（trie）。树中的每个**内部节点**代表一段共享前缀，占用若干物理页；每个**叶节点**代表一个具体请求的独有后缀部分。

- 每个节点 v 有**页开销** $c(v)$（存储该段 KV Cache 所需的物理页数）

- 每个**叶节点**（请求）有**服务收益** $w(v)$（如用户优先级、预期收入）；内部节点的收益为 0（共享前缀本身不产生收益，仅为请求服务）
- **依赖约束**：要服务某个请求（叶节点），必须分配从根到该叶的**整条路径**上所有节点的页
- 前缀共享：若两个请求共享某段前缀，该前缀的页**只需分配一次**

目标：在总物理页预算 P 内，选择一些请求服务，使总收益最大化。

(a) **树形 DP 状态定义**。设 $dp[v][j]$ 为以 v 为根的子树中，分配 j 个物理页时可获得的最大收益。注意：**选择子树中的任何叶节点，都必须先为节点 v 本身分配 $c(v)$ 页**。

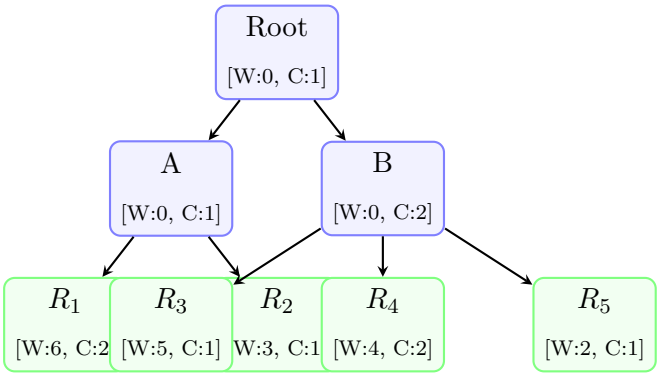
i. 写出叶节点（请求节点）的边界条件：

$$dp[v][j] = \begin{cases} ? & \text{if } j \geq c(v) \\ ? & \text{if } j < c(v) \end{cases}$$

ii. 写出内部节点 v（子节点集合为 $\mathrm{ch}(v) = \{u_1, u_2, \ldots\}$）的状态转移方程。

提示：先为 v 本身保留 $c(v)$ 页，剩余 $j - c(v)$ 页在各子树间分配。子树之间的页分配相互独立，等价于对子树做**分组背包合并**：依次合并每个子树的 DP 表。

(b) **手算例子**。下图为一个前缀树，共 5 个请求（叶节点 R_1–R_5）。每个节点标注为 $[w, c]$（收益，页开销）。物理页预算 $P = 7$。



例如，服务请求 R_1 需要分配路径 root $\to A \to R_1$ 共 $1+1+2=4$ 页；若同时服务 R_1 和 R_2，共享 root 和 A 的页，总共只需 $1+1+2+1=5$ 页。

i. 自底向上计算每个节点的 $dp[v][j]$ 表。先算叶节点 R_1–R_5，再合并算内部节点 A、B，最后算 root。

ii. 在预算 $P = 7$ 下，最优方案服务哪些请求？最大总收益是多少？

iii. 验证前缀共享带来的节省：若不共享前缀（每个请求独立分配从根到叶的全部页），服务同样的请求集合需要多少页？节省了多少？

(c) **复杂度分析**。

i. 合并两个子树的 DP 表（大小分别为 s_1 和 s_2）需要 $O(s_1 \cdot s_2)$ 时间。说明整棵树的总合并时间为 $O(nP^2)$，其中 n 为节点数。

ii. 在实际 LLM serving 中，n 可达数百（请求数 + 共享前缀数），P 可达数千（GPU 显存页数）。$O(nP^2)$ 是否实际可行？若不可行，提出一种降低复杂度的思路。（提示：如果大部分子树的 dp 表是稀疏的，可以怎样优化？）

标准答案：

(a) 树形 DP 状态定义 i. 叶节点（请求节点）的边界条件：

对叶节点 v（收益为 $w(v)$，页开销为 $c(v)$）：

$$dp[v][j] = \begin{cases} w(v) & \text{若} j \geq c(v) \quad \text{（分配足够的页，服务该请求）} \\ 0 & \text{若} j < c(v) \quad \text{（页不足，无法服务）} \end{cases}$$

ii. 内部节点的状态转移：

对内部节点 v（子节点 $\mathrm{ch}(v) = \{u_1, u_2, \ldots, u_k\}$）：

若 $j < c(v)$，则 $dp[v][j] = 0$（连节点 v 本身的页都无法分配）。

若 $j \geq c(v)$，先为 v 保留 $c(v)$ 页，剩余 $j - c(v)$ 页在各子树间分配。子树之间做**分组背包合并**：

初始化 $f_0[r] = 0$（$0 \leq r \leq j - c(v)$）。依次合并第 ℓ 个子树：

$$f_\ell[r] = \max_{0 \leq t \leq r} \left\{ f_{\ell-1}[r-t] + dp[u_\ell][t] \right\}$$

最终 $dp[v][j] = f_k[j - c(v)]$。

(b) 手算 i. 自底向上计算 dp 表：

叶节点（j 从 0 到 7）：

节点	$[w, c]$	j=0	j=1	j=2	j=3	j=4	j=5	j=6	j=7
R_1	$[6, 2]$	0	0	6	6	6	6	6	6
R_2	$[3, 1]$	0	3	3	3	3	3	3	3
R_3	$[5, 1]$	0	5	5	5	5	5	5	5
R_4	$[4, 2]$	0	0	4	4	4	4	4	4
R_5	$[2, 1]$	0	2	2	2	2	2	2	2

内部节点 A（$c(A)=1$，子节点 $\{R_1,R_2\}$）：

保留 1 页后，剩余 $j-1$ 页分配给 R_1 和 R_2。合并：

j	0	1	2	3	4	5	6	7
$dp[A][j]$	0	0	3	6	9	9	9	9

其中 $dp[A][2]=3$（仅服务 R_2），$dp[A][3]=6$（仅服务 R_1），$dp[A][4]=9$（同时服务 R_1 和 R_2：A=$1+R_1$=$2+R_2$=$1=4$ 页）。

内部节点 B（$c(B)=2$，子节点 $\{R_3,R_4,R_5\}$）：

保留 2 页后，剩余 $j-2$ 页分配给三个子节点。先合并 R_3 和 R_4，再与 R_5 合并：

j	0	1	2	3	4	5	6	7
$dp[B][j]$	0	0	0	5	7	9	11	11

其中 $dp[B][3]=5$（仅 R_3），$dp[B][4]=7$（R_3+R_5），$dp[B][5]=9$（R_3+R_4），$dp[B][6]=11$（R_3+R_4+R_5）。

根节点 Root（$c(\text{Root})=1$，子节点 $\{A,B\}$）：

保留 1 页后，剩余 $j-1$ 页分配给 A 和 B 子树。合并：

j	0	1	2	3	4	5	6	7
$dp[\text{Root}][j]$	0	0	0	3	6	9	9	11

$dp[\text{Root}][7]=11$，有两种最优方案取到：

- $dp[A][0]+dp[B][6]=0+11=11$：服务 $\{R_3,R_4,R_5\}$
- $dp[A][3]+dp[B][3]=6+5=11$：服务 $\{R_1,R_3\}$

ii. 预算 $P=7$ 下，最优方案（任选一个）：

方案一：服务 $\{R_3,R_4,R_5\}$，页分配：Root=1, B=2, R_3=1, R_4=2, R_5=1，共 7 页。总收益 $=5+4+2=\boxed{11}$。

方案二：服务 $\{R_1,R_3\}$，页分配：Root=1, A=1, R_1=2, B=2, R_3=1，共 7 页。总收益 $=6+5=11$。

iii. 前缀共享带来的节省：

以方案一（服务 $\{R_3,R_4,R_5\}$）为例：

- 不共享前缀：R_3 需 Root$\to B\to R_3$ 共 1+2+1 = 4 页；R_4 需 1+2+2 = 5 页；R_5 需 1+2+1 = 4 页。总计 4+5+4 = 13 页。
- 共享前缀：Root 和 B 的页只分配一次，共 1+2+1+2+1 = 7 页。
- **节省**：$13-7=6$ 页（节省 46%）。

(c) 复杂度分析 **i.** 合并两个子树的 DP 表（大小分别为 s_1 和 s_2）需要 $O(s_1 \cdot s_2)$ 时间。对于每个内部节点 v，依次合并其 k_v 个子树，每次合并的表大小上界为 P，因此节点 v 的合并时间为 $O(k_v \cdot P^2)$。

对所有节点求和：$\sum_v k_v \cdot P^2 = P^2 \sum_v k_v$。由于 $\sum_v k_v = n-1$（树的总边数），总合并时间为 $O((n-1) \cdot P^2) = \boxed{O(nP^2)}$。

ii. 当 n 达数百、P 达数千时，$O(nP^2) \approx 100 \times 1000^2 = 10^8$，在现代硬件上勉强可行但较慢。若 P 更大则不实际。

降低复杂度的思路：大部分子树所需的总页数远小于 P，其 DP 表中大量条目为零或不变（稀疏性）。可以：

- 使用**稀疏表示**：只存储和合并非零/有变化的 DP 条目，避免遍历整个 $[0, P]$ 范围。
- 对 DP 表进行**页数离散化/舍入**，降低表的有效大小。
- 对子树按大小排序合并，利用”小子树优先”的策略使得中间表大小增长缓慢，总合并复杂度可降至 $O(nP \log n)$。

第七题：0-1 背包的 FPTAS——大模型推理预算分配

大模型背景：LLM Serving 中的请求选择

在大规模 LLM 服务中，当系统过载时，需要从等待队列中选择一批请求处理，使得在有限的计算预算（GPU 算力或 KV Cache 容量）内，总服务质量最高。每个请求的计算开销（token 数）和优先级（SLA 等级、付费用户权重）不同，这本质上是一个 **0-1 背包问题**。精确 DP 的复杂度 $O(nW)$ 在预算 W 很大时不可行，而 **FPTAS**（全多项式时间近似方案）通过值域缩放，能在 $O(n^2/\varepsilon)$ 时间内找到 $(1-\varepsilon)$ 近似解。

(a) **伪多项式时间与 NP 难的关系**。0-1 背包的精确 DP 复杂度为 $O(nW)$。

i. 若输入中 W 用二进制表示，其编码长度为 $\lceil \log_2 W \rceil$ 位。说明 $O(nW) = O(n \cdot 2^{\lceil \log_2 W \rceil})$，即对输入编码长度而言是**指数级**的。这就是”伪多项式时间”的含义。

ii. 0-1 背包是**弱 NP 难**问题。解释”弱 NP 难”与”强 NP 难”的区别，并各举一个例子。

(b) **FPTAS 算法描述**。给定 $\varepsilon > 0$，FPTAS 通过以下步骤将 0-1 背包转化为多项式时间可解的问题：

i. 令 $V_{\max} = \max_i v_i$，缩放因子 $K = \dfrac{\varepsilon \cdot V_{\max}}{n}$。对每个物品计算缩放后的价值 $v'_i = \left\lfloor \dfrac{v_i}{K} \right\rfloor$。说明每个 v'_i 的取值范围（用 n, ε 表示上界）。

ii. 对缩放后的问题，运行**最小重量 DP**：$dp[v]$ 表示恰好达到总缩放价值 v 时的最小总重量。写出 $dp[v]$ 的状态转移方程和初始条件。

iii. DP 表的大小（v 的上界）是多少？（用 n, ε 表示）总时间复杂度是多少？验证这是关于 n 和 $1/\varepsilon$ 的多项式。

(c) **近似比证明**。设 OPT 为精确最优值，ALG 为 FPTAS 的输出值。

核心要求：待证命题

$\mathrm{ALG} \geq (1-\varepsilon)\,\mathrm{OPT}$。

提示：

i. 设 S^* 为最优解的物品集合。说明每个物品取整损失 $v_i - K \cdot v'_i < K$，因此总损失 $\mathrm{OPT} - K \cdot \sum_{i \in S^*} v'_i < n \cdot K = \varepsilon \cdot V_{\max}$。

ii. 说明 $V_{\max} \leq \mathrm{OPT}$（最优解至少包含一个物品），从而总损失 $< \varepsilon \cdot \mathrm{OPT}$。

iii. 设 FPTAS 的解为 S'，在缩放问题中 S' 是最优的。综合以上两步，完成证明：$\mathrm{ALG} \geq K \cdot \sum_{i \in S'} v'_i \geq K \cdot \sum_{i \in S^*} v'_i \geq (1-\varepsilon)\,\mathrm{OPT}$。

(d) **数值手算**。 5 个推理请求，计算预算 $W = 11$：

请求 i	计算开销 w_i	服务收益 v_i	v_i/K	缩放后 v'_i
1	5	35		
2	2	155		
3	6	50		
4	4	250		
5	2	30		

取 $\varepsilon = 0.4$。

i. 计算 $V_{\max}$、K、以及每个物品的缩放价值 v'_i（填入上表）。DP 表的大小（v' 的上界 $\sum_i v'_i$）是多少？

ii. 运行缩放后的最小重量 DP。提示：不需要填写全部表格，只需追踪 $dp[v']$ 中能够被 $W = 11$ 容纳的那些状态。找到 FPTAS 的近似解：选了哪些请求？近似总收益 ALG 是多少？

iii. 用暴力枚举($2^5 = 32$ 种子集)求出精确最优值 OPT 及最优方案。计算 ALG/OPT，验证是否 $\geq 1 - \varepsilon = 0.6$。

iv. 观察：FPTAS 的解与精确最优解有何不同？造成这种差异的原因是什么？（提示：比较两个方案在缩放后的价值 $\sum v'_i$ 和实际价值 $\sum v_i$。）

标准答案：

(a) 伪多项式时间与 NP 难的关系 **i.** W 用二进制表示时，编码长度为 $L = \lceil \log_2 W \rceil$ 位。因此 $W = \Theta(2^L)$，故

$$O(nW) = O(n \cdot 2^{\lceil \log_2 W \rceil})$$

这关于输入编码长度 L 是**指数级**的。虽然 $O(nW)$ "看起来" 是多项式（如果把 W 当作数值大小），但从计算复杂性理论的角度，输入大小包含 $\log W$ 而非 W。因此 $O(nW)$ 是**伪多项式时间**——它是输入**数值**的多项式，但是输入**编码长度**的指数。

ii. 弱 NP 难 vs 强 NP 难：

- **弱 NP 难**：问题是 NP 难的，但存在伪多项式时间算法。当输入中的数值被多项式限界时（$W = O(\text{poly}(n))$），问题变为多项式时间可解。**例子**：0-1 背包问题（$O(nW)$）。
- **强 NP 难**：即使输入中的所有数值被多项式限界，问题仍然是 NP 难的。不存在伪多项式时间算法（除非 $P = NP$）。**例子**：3-划分问题（3-Partition）、旅行商问题（TSP）的一般形式。

(b) FPTAS 算法描述 **i. 缩放后价值的取值范围**：

$v_i' = \left\lfloor \frac{v_i}{K} \right\rfloor \le \frac{v_i}{K} = \frac{v_i \cdot n}{\varepsilon \cdot V_{\max}} \le \frac{V_{\max} \cdot n}{\varepsilon \cdot V_{\max}} = \frac{n}{\varepsilon}$

因此每个 $v_i' \in \{0, 1, \ldots, \lfloor n/\varepsilon \rfloor\}$。

ii. 最小重量 DP：

定义 $dp[v]$ 为恰好达到总缩放价值 v 时的最小总重量。

初始条件：$dp[0] = 0$，$dp[v] = +\infty$（$v > 0$）。

对每个物品 $i = 1, \ldots, n$，从大到小更新：

$$dp[v] \leftarrow \min\big(dp[v],\ dp[v - v_i'] + w_i\big), \quad v = V', V'-1, \ldots, v_i'$$

最终答案：$\max\{v : dp[v] \le W\}$，对应的物品集合即为近似解。

iii. DP 表大小与时间复杂度：

v 的上界 $V' = \sum_{i=1}^{n} v_i' \le n \cdot \lfloor n/\varepsilon \rfloor \le n^2/\varepsilon$。

总时间 $= O(n \cdot V') = O(n \cdot n^2/\varepsilon) = \boxed{O(n^3/\varepsilon)}$，是关于 n 和 $1/\varepsilon$ 的多项式。

(c) 近似比证明

证明. **i.** 设 S^* 为精确最优解的物品集合。对每个物品 i，由取整定义：

$$v_i' = \left\lfloor \frac{v_i}{K} \right\rfloor \implies v_i - K v_i' < K$$

对 S^* 中所有物品求和：

$$\text{OPT} - K \sum_{i \in S^*} v_i' = \sum_{i \in S^*} (v_i - K v_i') < |S^*| \cdot K \le n \cdot K = \varepsilon \cdot V_{\max}$$

ii. 精确最优解至少包含一个物品（否则 $\mathrm{OPT}=0$，近似比平凡成立），且默认所有物品的容量都小于等于背包容量，因此 $\mathrm{OPT}\geq V_{\max}$。

故 $\varepsilon\cdot V_{\max}\leq\varepsilon\cdot\mathrm{OPT}$，结合上式得：

$$\mathrm{OPT}-K\sum_{i\in S^*}v_i'<\varepsilon\cdot\mathrm{OPT}$$

iii. 设 FPTAS 的解为 S'。S' 在缩放问题中是最优的（最大化 $\sum v_i'$ 且满足重量约束），因此 $\sum_{i\in S'}v_i'\geq\sum_{i\in S^*}v_i'$（$S^*$ 也满足重量约束，故在缩放问题中也可行）。

综合：

$$\begin{aligned}\mathrm{ALG}=\sum_{i\in S'}v_i&\geq K\sum_{i\in S'}v_i'\quad(v_i\geq Kv_i')\\&\geq K\sum_{i\in S^*}v_i'\quad(S'\text{ 在缩放问题中最优})\\&>\mathrm{OPT}-\varepsilon\cdot\mathrm{OPT}=(1-\varepsilon)\,\mathrm{OPT}\end{aligned}$$

□

(d) 数值手算 i. 计算 $V_{\max}$、K 和缩放价值：

$V_{\max}=\max\{35,155,50,250,30\}=250$

$K=\dfrac{\varepsilon\cdot V_{\max}}{n}=\dfrac{0.4\times250}{5}=20$

请求 i	计算开销 w_i	服务收益 v_i	v_i/K	缩放后 v_i'
1	5	35	1.75	1
2	2	155	7.75	7
3	6	50	2.5	2
4	4	250	12.5	12
5	2	30	1.5	1

DP 表的大小：$\sum_i v_i'=1+7+2+12+1=23$，即 v' 的上界为 23。

ii. 运行最小重量 DP：

初始化 $dp[0]=0$，其余为 $+\infty$。依次处理各物品后（省略中间步骤），满足 $dp[v']\leq W=11$ 的关键可行状态为：

v'	0	1	7	8	12	13	19	20
$dp[v']$	0	2	2	4	4	6	6	8

最大可行缩放价值：$v'=20$（$dp[20]=8\leq11$）。

回溯：$v'=20$ 由物品 5（$v_5'=1,w_5=2$）从 $dp[19]=6$ 转移而来；$dp[19]=6$ 由物品 4（$v_4'=12,w_4=4$）从 $dp[7]=2$ 转移；$dp[7]=2$ 由物品 2（$v_2'=7,w_2=2$）从 $dp[0]=0$ 转移。

FPTAS 解：$S' = \{2, 4, 5\}$，总重量 $= 2 + 4 + 2 = 8 \leq 11$。

近似总收益：$\text{ALG} = v_2 + v_4 + v_5 = 155 + 250 + 30 = \boxed{435}$。

iii. 暴力枚举精确最优：

在 $2^5 = 32$ 种子集中，满足 $\sum w_i \leq 11$ 的最优子集为 $\{1, 2, 4\}$（$w = 5 + 2 + 4 = 11$，$v = 35 + 155 + 250 = 440$）。

$$\text{OPT} = 440, \quad \text{最优方案} S^* = \{1, 2, 4\}$$

验证：$\text{ALG}/\text{OPT} = 435/440 = 87/88 \approx 0.989 \geq 1 - \varepsilon = 0.6$。✓

iv. 差异分析：

FPTAS 选了 $\{2, 4, 5\}$（价值 435），而精确最优为 $\{1, 2, 4\}$（价值 440）。两者的**缩放后价值**相同（$\sum v'_i = 7 + 12 + 1 = 20$），但实际价值不同。

原因：物品 1（$v_1 = 35, v'_1 = 1$）和物品 5（$v_5 = 30, v'_5 = 1$）在缩放后有**相同**的价值（均为 1），但实际价值不同（35 vs 30）。由于物品 5 更轻（$w_5 = 2 < w_1 = 5$），最小重量 DP 优先选择了物品 5。取整丢失了 35 和 30 之间的区分度（两者除以 $K = 20$ 后取整均为 1），这正是 FPTAS 近似误差的来源。

第八题：贪心算法——Huffman 编码与词表压缩

大模型背景：LLM 词表压缩与 Zipf 定律

大语言模型的词表通常包含数万个 token，且其频率分布遵循 **Zipf 定律**（极度不均匀）：少数高频词出现极为频繁，大量低频词极少出现。Huffman 编码可利用这种不均匀性压缩存储空间，是模型量化与压缩的理论基础之一。

(a) 给定 6 个高频 token 及其在语料中出现的相对频率（单位：千次）：

Token	`the`	`a`	`is`	`of`	`in`	`to`
频率	45	13	12	16	9	5

按 Huffman 算法构造最优前缀编码树，画出 Huffman 树（标注每条边的 0/1），并写出每个 token 的编码。

(b) 计算：

i. Huffman 编码的**平均编码长度**（按频率加权）；

ii. 若使用等长编码（$\lceil \log_2 6 \rceil$ 位），平均编码长度是多少？

iii. Huffman 编码相比等长编码节省了多少比例的空间？

(c) 用**交换论证法（exchange argument）**证明 Huffman 算法的贪心选择性质：

核心要求：待证命题

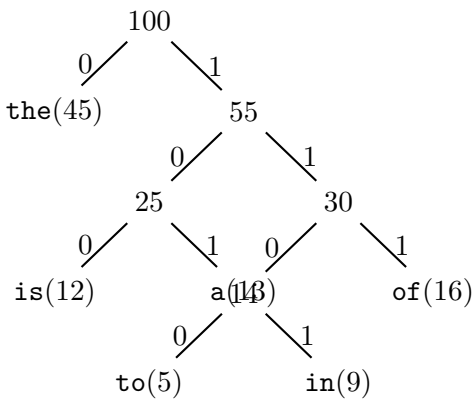
在最优前缀编码树中，频率最低的两个符号一定是兄弟节点（位于树的最深层）。

提示：假设最优树中频率最低的符号 x, y 不是深度最大的兄弟节点，构造一棵交换后的树，证明交换不会增加（反而可能减少）平均编码长度，从而导出矛盾。

标准答案：

(a) 构造 Huffman 树 频率排序：`to`(5), `in`(9), `is`(12), `a`(13), `of`(16), `the`(45)。
按 Huffman 算法逐步合并频率最小的两个节点：

1. 合并 `to`(5) 和 `in`(9) → 节点 $N_1(14)$。队列：`is`(12), `a`(13), $N_1(14)$, `of`(16), `the`(45)
2. 合并 `is`(12) 和 `a`(13) → 节点 $N_2(25)$。队列：$N_1(14)$, `of`(16), $N_2(25)$, `the`(45)
3. 合并 $N_1(14)$ 和 `of`(16) → 节点 $N_3(30)$。队列：$N_2(25)$, $N_3(30)$, `the`(45)
4. 合并 $N_2(25)$ 和 $N_3(30)$ → 节点 $N_4(55)$。队列：`the`(45), $N_4(55)$
5. 合并 `the`(45) 和 $N_4(55)$ → 根节点 (100)



每个 token 的编码：

Token	编码	长度
`the`	0	1
`is`	100	3
`a`	101	3
`to`	1100	4
`in`	1101	4
`of`	111	3

(b) 编码长度计算 **i. Huffman 编码的平均编码长度**（总频率 $= 45+13+12+16+9+5 = 100$）：

$$\bar{L}_H = \frac{45\times1 + 12\times3 + 13\times3 + 5\times4 + 9\times4 + 16\times3}{100} = \frac{45+36+39+20+36+48}{100} = \frac{224}{100} = \boxed{2.24\text{ 比特}}$$

ii. 等长编码：$\lceil \log_2 6 \rceil = 3$ 位，平均编码长度 $= \boxed{3\text{ 比特}}$。

iii. 节省比例：

$$\frac{3-2.24}{3} = \frac{0.76}{3} \approx \boxed{25.3\%}$$

(c) 交换论证法证明贪心选择性质

证明. 设 T 是任意最优前缀编码树，x 和 y 是频率最低的两个符号（$f_x \le f_y$）。

设 a 和 b 是 T 中深度最大的一对兄弟叶节点（任何二叉树中，最深层必存在兄弟叶节点）。

记 $d(c)$ 为符号 c 在树中的深度。加权路径长度为 $B(T) = \sum_c f_c \cdot d(c)$。

步骤 1：交换 x 和 a（若 $x \ne a$）。代价变化为：

$$\Delta_1 = (f_x - f_a)\big(d(a) - d(x)\big)$$

由于 a 在最深层，$d(a) \ge d(x)$；x 频率最低，$f_x \le f_a$。因此 $\Delta_1 \le 0$。

步骤 2：在交换后的树中，交换 y 和 b（若 $y \ne b$）。类似地：

$$\Delta_2 = (f_y - f_b)\big(d(b) - d(y)\big) \le 0$$

两次交换后，x 和 y 成为最深层的兄弟节点，新树 T' 满足：

$$B(T') = B(T) + \Delta_1 + \Delta_2 \le B(T)$$

由于 T 是最优的，$B(T') \ge B(T)$。因此 $B(T') = B(T)$，即 T' 也是最优树。

这证明了**存在一棵最优前缀编码树**，其中频率最低的两个符号是最深层的兄弟节点——这正是 Huffman 算法在第一步的贪心选择。 □

第九题：贪心调度——GPU 推理请求的最小最大延迟

大模型背景：大模型推理服务的请求调度

在生产环境中，推理服务器（如 vLLM、TensorRT-LLM）同时接收大量用户请求，每个请求有不同的处理时间（由序列长度决定）和截止时间（用户的 SLA 要求）。合理调度请求顺序，可以最小化用户的最大等待超时时间。

(a) 设有 n 个推理请求，请求 i 的处理时间为 t_i，截止时间为 d_i。所有请求在时刻 0 均已到达，每次只处理一个请求（单 GPU）。定义请求 i 的**延迟**为 $L_i = C_i - d_i$，其中 C_i 为其完成时刻；若 $C_i \le d_i$ 则延迟为负（未超时）。目标是最小化最大延迟 $L_{\max} = \max_i L_i$。

证明：**最早截止时间优先（EDF）策略**（按 d_i 升序调度）最小化 $L_{\max}$。

提示：用交换论证法——若某调度 σ 中存在相邻两个请求 i, j 满足 $d_i > d_j$ 但 i 排在 j 之前，证明交换它们不会增大 $L_{\max}$。

(b) 对如下 4 个推理请求，按 EDF 策略调度，计算每个请求的完成时刻、延迟，以及最大延迟 $L_{\max}$：

请求	t_i（处理时间）	d_i（截止时间）	C_i（完成时刻）	L_i（延迟）
1	2	4		
2	1	2		
3	3	7		
4	2	5		

标准答案：

(a) 证明 EDF 策略最小化最大延迟

问题描述设有 n 个推理请求，每个请求 i 的处理时间为 $t_i > 0$，截止时间为 d_i。所有请求在时刻 0 同时到达，系统每次只能处理一个请求（单 GPU）。定义请求 i 的完成时刻为 C_i，延迟为 $L_i = C_i - d_i$。目标是最小化最大延迟 $L_{\max} = \max_i L_i$。

EDF 策略：按照截止时间非递减的顺序调度，即 d_i 较小的请求优先执行。若截止时间相同，顺序任意。

交换论证证明设 σ 为任意一个调度方案。若 σ 中存在相邻的两个请求 i 和 j，其中 i 排在 j 之前，但 $d_i > d_j$（即违反 EDF 顺序），则交换 i 与 j 的位置得到新调度 σ'，我们证明交换后 $L_{\max}$ 不会增大。

记 i 的开始时刻为 s（即 i 开始前已累积的处理时间）。交换前：

$$C_i = s + t_i, \quad L_i = s + t_i - d_i,$$

$$C_j = s + t_i + t_j, \quad L_j = s + t_i + t_j - d_j.$$

交换后（j 先执行）：

$$C'_j = s + t_j, \quad L'_j = s + t_j - d_j,$$

$$C'_i = s + t_j + t_i, \quad L'_i = s + t_i + t_j - d_i.$$

比较交换前后的延迟：

- $L'_j = s + t_j - d_j < s + t_i + t_j - d_j = L_j$，因为 $t_i > 0$，所以 j 的延迟严格减小。
- $L'_i = s + t_i + t_j - d_i < s + t_i + t_j - d_j = L_j$，因为 $d_i > d_j$，所以 i 的延迟小于交换前 j 的延迟。

因此 $\max(L'_i, L'_j) < L_j \leq L_{\max}(\sigma)$。其他请求的完成时间不受影响，故 $L_{\max}(\sigma') \leq L_{\max}(\sigma)$。

通过反复交换所有相邻的逆序对，可以将任意调度转换为 EDF 调度（即按 d_i 非递减排列），且每次交换均不增加 $L_{\max}$。因此 EDF 调度达到最小可能的最大延迟。

(b) EDF 调度实例计算

给定 4 个推理请求：

请求	处理时间 t_i	截止时间 d_i
1	2	4
2	1	2
3	3	7
4	2	5

按 d_i 升序排列：请求 2 ($d = 2$)，请求 1 ($d = 4$)，请求 4 ($d = 5$)，请求 3 ($d = 7$)。
调度过程（初始时刻 0）：

顺序	请求	t_i	开始时间	完成时间 C_i	延迟 $L_i = C_i - d_i$
1	2	1	0	1	1 - 2 = -1
2	1	2	1	3	3 - 4 = -1
3	4	2	3	5	5 - 5 = 0
4	3	3	5	8	8 - 7 = 1

最大延迟为 $L_{\max} = \max\{-1, -1, 0, 1\} = \boxed{1}$。
最大延迟为 1，由请求 3 产生（完成时刻 8，截止时间 7，超时 1 个单位）。

第十题：FlashAttention——分块、IO 复杂度与在线 softmax

大模型背景：FlashAttention：IO 感知的注意力算法

标准注意力需要将 $n \times n$ 的注意力矩阵 $S = QK^\top/\sqrt{d}$ 物化到 GPU 的 **HBM**（高带宽显存，容量大但带宽有限），再读回做 softmax 和乘 V。当序列长度 n 很大时，这 $O(n^2)$ 的 HBM 读写成为瓶颈。**FlashAttention**（Dao et al., 2022）采用分块策略：将 Q, K, V 按序列维度分成大小为 B 的块，在片上 **SRAM**（容量小但速度极快）中逐块计算注意力，并利用**在线 softmax** 技巧实时修正归一化，从而**无需物化完整注意力矩阵**，将额外显存从 $O(n^2)$ 降至 $O(n)$。

(a) **分块策略与显存分析**。标准注意力的中间结果 $S \in \mathbb{R}^{n\times n}$ 需要 $O(n^2)$ 显存。FlashAttention 将 Q 分成 $T_Q = \lceil n/B_r \rceil$ 个块，K, V 分成 $T_{KV} = \lceil n/B_c \rceil$ 个块，每次在 SRAM 中只存储一个 $B_r \times B_c$ 的局部注意力块。

i. 每次 SRAM 中需要存储的数据包括：一个 Q 块（$B_r \times d$）、一个 K 块（$B_c \times d$）、一个 V 块（$B_c \times d$）、局部注意力块（$B_r \times B_c$）、以及输出累加器（$B_r \times d$）。写出 SRAM 占用量关于 B_r, B_c, d 的表达式。若 $B_r = B_c = B$，SRAM 大小 M 需要满足什么条件？

ii. 解释为何 FlashAttention 的**额外显存**（不含输入 Q, K, V 和最终输出 O）只有 $O(n)$ 而非 $O(n^2)$。这 $O(n)$ 具体存储的是什么？

(b) **IO 复杂度分析**。 FlashAttention 的核心收益在于减少 HBM 读写次数。

i. 标准注意力需要：(1) 从 HBM 读 Q, K 计算 $S = QK^\top$，将 S 写回 HBM；(2) 从 HBM 读 S 做 softmax，将 $P = \mathrm{softmax}(S)$ 写回；(3) 从 HBM 读 P, V 计算输出。写出标准注意力的总 HBM 读写量（用 n, d 表示，忽略常数）。

ii. FlashAttention 的外层循环遍历 T_{KV} 个 KV 块，内层循环遍历 T_Q 个 Q 块。每次内层迭代从 HBM 读取一个 Q 块和对应的输出块，写回更新后的输出块。每次外层迭代读取一个 KV 块。写出总 HBM 读写量关于 n, d, B_r, B_c 的表达式。

iii. 令 $B_r = B_c = B = \Theta(M/d)$（SRAM 约束），将 HBM 读写量用 n, d, M 表示。当 $M = \Theta(nd)$（SRAM 足够大）时，IO 量为多少？与标准注意力的 $O(n^2 d)$ 相比改善了多少？

(c) **在线 softmax 与经典分治的对比**。

分块计算 $\mathrm{softmax}(QK^\top) \cdot V$ 的核心难点在于：softmax 需要对第 i 个 query 的**全部** n 个 key 的得分做归一化，但每个块只看到 B_c 个 key。

i. 设前 j 个 K 块处理完后，第 i 个 query 已累积的局部结果为 $\tilde{o}_i^{(j)} = \sum_{k \le jB_c} e^{s_{ik} - m_i^{(j)}} v_k$ 和对应的 log-sum-exp $L_i^{(j)} = \log \sum_{k \le jB_c} e^{s_{ik} - m_i^{(j)}}$，其中 $m_i^{(j)}$ 为当前已见得分的最大值。当处理第 j+1 个 K 块后，新的最大值可能更新为 $m_i^{(j+1)} > m_i^{(j)}$。写出 $\tilde{o}_i^{(j+1)}$ 的更新公式（需要对旧的 $\tilde{o}_i^{(j)}$ 做什么修正?）。

ii. 经典分治（如 Merge Sort）的子问题**独立**求解后合并，合并时不修改子问题的内部结果。FlashAttention 的在线 softmax 与此有何本质区别？这种区别对 GPU 并行性有什么影响?

标准答案：

(a) 分块策略与显存分析 i. SRAM 占用量：

每次迭代在 SRAM 中需要存储：

- Q 块：$B_r \times d$
- K 块：$B_c \times d$
- V 块：$B_c \times d$
- 局部注意力块 S_{local}：$B_r \times B_c$
- 输出累加器 O_{local}：$B_r \times d$

SRAM 占用量为：

$$\boxed{2B_r d + 2B_c d + B_r B_c}$$

若 $B_r = B_c = B$，则占用 $4Bd + B^2$。SRAM 大小 M 需满足：

$$\boxed{B^2 + 4Bd \le M}$$

ii. 额外显存为 $O(n)$ 的原因：

FlashAttention **不物化**完整的 $n \times n$ 注意力矩阵。每个 $B_r \times B_c$ 的局部注意力块在 SRAM 中计算并立即用于更新输出，随后被丢弃。

额外显存 $O(n)$ 具体存储的是每个 query 行的辅助统计量：

- 行最大值 m_i：n 个标量，用于数值稳定的在线 softmax
- 行 log-sum-exp L_i：n 个标量，用于归一化

因此额外显存仅为 $O(n)$，而非标准注意力的 $O(n^2)$。

(b) IO 复杂度分析　i. 标准注意力的 HBM 读写量：

1. 计算 $S = QK^\top/\sqrt{d}$：读 Q（nd）和 K（nd），写 S（n^2）。IO：$O(nd + n^2)$
2. Softmax：读 S（n^2），写 P（n^2）。IO：$O(n^2)$
3. 计算 $O = PV$：读 P（n^2）和 V（nd），写 O（nd）。IO：$O(n^2 + nd)$

总 HBM 读写量：$\boxed{\Theta(nd + n^2)}$。

ii. FlashAttention 的 HBM 读写量：

外层循环遍历 $T_{KV} = \lceil n/B_c \rceil$ 个 KV 块，每次读一个 K 块（$B_c d$）和 V 块（$B_c d$）。内层循环遍历 $T_Q = \lceil n/B_r \rceil$ 个 Q 块，每次读 Q 块（$B_r d$）和输出块（$B_r d$），写更新后的输出块（$B_r d$）。

总 IO：

$$\begin{aligned}\text{KV 读取} &= T_{KV} \times 2B_c d = 2nd \\ \text{Q/O 读写} &= T_{KV} \times T_Q \times 3B_r d = \frac{n}{B_c} \cdot \frac{n}{B_r} \cdot 3B_r d = \frac{3n^2 d}{B_c}\end{aligned}$$

$$\boxed{\text{总 HBM IO} = O\left(\frac{n^2 d}{B_c} + nd\right)}$$

iii. 令 $B_c = \Theta(M/d)$（由 SRAM 约束 $B^2 + 4Bd \le M$ 得到，在 FlashAttention 的典型参数下 $B \ll d$ 即 $M \ll d^2$ ，此时 4Bd 项主导，$B \approx M/(4d)$）：

$$\text{IO} = O\left(\frac{n^2 d}{M/d} + nd\right) = O\left(\frac{n^2 d^2}{M} + nd\right)$$

当 $M = \Theta(nd)$ 时：

$$\text{IO} = O\left(\frac{n^2 d^2}{nd} + nd\right) = O(nd + nd) = \boxed{O(nd)}$$

与标准注意力的 $\Theta(nd + n^2)$ 相比，当 $n \gg d$ 时，IO 从 $O(n^2)$ 降至 $O(nd)$，改善了 $O(n/d)$ 倍。对于典型的长序列场景（如 $n = 8192, d = 128$），这意味着约 64× 的 IO 减少。

(c) 在线 softmax 与经典分治的对比　i. $\tilde{o}_i^{(j+1)}$ 的更新公式：

当处理第 j+1 个 K 块后，新的最大值 $m_i^{(j+1)} = \max\left(m_i^{(j)},\ m_i^{\text{new}}\right)$，其中 m_i^{new} 是新块中得分的最大值。

如果 $m_i^{(j+1)} > m_i^{(j)}$，需要对旧的累积结果进行**重新缩放**（rescaling）：

$$\boxed{\tilde{o}_i^{(j+1)} = e^{m_i^{(j)} - m_i^{(j+1)}} \cdot \tilde{o}_i^{(j)} + \sum_{k \in \text{new block}} e^{s_{ik} - m_i^{(j+1)}} v_k}$$

旧的 $\tilde{o}_i^{(j)}$ 需要乘以修正因子 $e^{m_i^{(j)}-m_i^{(j+1)}}$（$\leq 1$），以将其从旧的最大值基准调整到新的最大值基准。同时，记线性归一化项 $\ell_i^{(j)} := \sum_{k\leq jB_c} e^{s_{ik}-m_i^{(j)}}$（即 $L_i^{(j)} = \log \ell_i^{(j)}$），则 $\ell_i^{(j)}$ 也需类似修正：

$$\ell_i^{(j+1)} = e^{m_i^{(j)}-m_i^{(j+1)}} \cdot \ell_i^{(j)} + \sum_{k\in\text{new block}} e^{s_{ik}-m_i^{(j+1)}}$$

ii. 与经典分治的本质区别：

经典分治（如 Merge Sort）中，子问题**独立**求解，合并时只读取子问题的结果进行组合，**不修改**子问题已经计算好的内部结果。

FlashAttention 的在线 softmax 则不同：当处理新的 K 块时，可能需要**回溯修正**之前所有块累积的中间结果（通过 rescaling 因子 $e^{m^{\text{old}}-m^{\text{new}}}$）。这意味着后续块的处理会**修改**先前块的贡献，子问题之间存在**单向依赖**。

对 GPU 并行性的影响：这种依赖关系意味着不同 K 块的处理**不能完全并行**——必须按顺序处理各 K 块以维护正确的 running max 和 running sum。然而，同一个 K 块内的不同 query 行之间仍然是独立的，可以完全并行。因此 FlashAttention 在 query 维度上并行（T_Q 个块同时处理），但在 KV 维度上是顺序的，这是在 IO 效率和并行性之间的一个权衡。